



```

<web-app>
  <servlet>
    <servlet-name>jersey-servlet</servlet-name>
    <servlet-class>
      com.sun.jersey.spi.container.servlet.ServletContainer</servlet-class>
    <init-param>
      <param-name>
        com.sun.jersey.config.property.packages</param-name>
      <param-value>webservice</param-value>
    </init-param>
  </servlet>
  <servlet-mapping>
    <servlet-name>jersey-servlet</servlet-name>
    <url-pattern>/ws-rest/*</url-pattern>
  </servlet-mapping></web-app>
  
```

<servlet-class>package.class</servlet-class>

➤ conf :

- server.xml : contient les éléments de configuration du serveur ;
- context.xml : contient les directives communes à toutes les applications web déployées sur le serveur ;
- tomcat-users.xml : contient entre autres l'identifiant et le mot de passe permettant d'accéder à l'interface d'administration de votre serveur Tomcat ;
- web.xml : contient les paramètres de configuration communs à toutes les applications web déployées sur le serveur

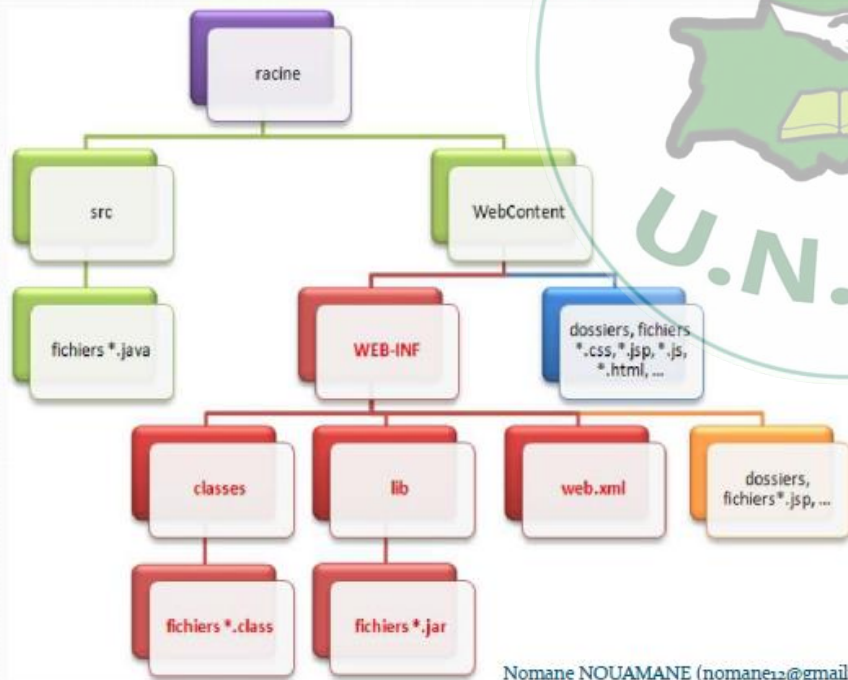
➤ Bin

- Contient les scripts permettant de démarrer/arrêter tomcat
- Webapp
- Le dossier par défaut qui contient les applications déployées sur le serveur

➤ Logs

- Contient les traces d'exécution du serveur. Si vous avez un problème consulter ce dossier pour en savoir davantage

Structure d'une application Java EE sous Eclipse



Nomane NOUAMANE (nomane12@gmail.com)



HTTP Methods:

- GET: is used to request data from specified resource
Ex: path/project?param=value¶m2=val2
- POST: is used to send data to a server to create/update a resource
- PUT: is used to send data to a server to create/update a resource
- HEAD: is almost identical to get, but without the response body.
- DELETE the specified resource
- OPTION: describe the communication options for the target resource

La classe HttpServlet

- ☐ Fournit le cadre pour l'implémentation des servlets reposant sur le protocole HTTP.
- ☐ service() : pour le traitement de toute requête HTTP
- ☐ doGet() : pour la méthode "GET"
- ☐ doHead() : pour la méthode "HEAD"
- ☐ doPost() : pour la méthode "POST"
- ☐ doPut() : pour la méthode "PUT"
- ☐ doDelete() : pour la méthode "DELETE"

On ne peut pas utiliser la même méthode HTTP avec des différentes fonctions dans le même path

Une fonction pour qu'elle soit accessible doit obligatoirement être public

Code de statut (famille de code HTTP) :

- 100-199 : retourne une réponse provisoire
- 200-299 : Succès de la requête
- 300-399 : Codes à la redirection
- 400-499 : Erreur dans la requête
- 500-599 : Erreur serveur, il n'a pas pu traiter la requête

```
private String id,name;

public String getId() {
    return id;
}

public void setId(String id) {
    this.id = id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public User(String id,String name){
    this.name=name;
    this.id=id;
}

}
```

```
import java.util.HashMap;
import java.util.Map;
```

```
import javax.ws.rs.Consumes;
import javax.ws.rs.DELETE;
import javax.ws.rs.GET;
import javax.ws.rs.PUT;
import javax.ws.rs.Path;
import javax.ws.rs.Produces;
import javax.ws.rs.QueryParam;
import javax.ws.rs.core.MediaType;
```

```
@Path("/exam")
public class WebCC {

    public static Map<String,User> users=new HashMap<>();

    //Ajouter user
    @PUT
    // @Consumes(MediaType.APPLICATION_JSON)
    public String addUser(@QueryParam("name") String name,@QueryParam("id") String id){
        users.put(id, new User(id,name));
        return "un utilisateur a ete ajoute avec succee";
    }

    //Supprimer un utilisateur
    @DELETE
    // @Consumes(MediaType.APPLICATION_JSON)
    public String SupUser(@QueryParam("id") String id){
        users.remove(id);
        return "removed";
    }

    //Afficher la liste de tous les utilisateurs
    @GET
    @Produces("application/json")
    @Path("/s")
    public Map ListeUsers(){
        return users;
    }

    //Afficher un utilisateur
    @GET
    public User UserInfo(@QueryParam("id") String id){
        return users.get(id);
    }
}
```

JSON: {"param": "val", "prm2": "val2"}



deux paramètres :

request : cet objet contient la requête et permet d'avoir accès à toutes ses informations, telles que les en-têtes (headers) et le corps de la requête.

* **HttpServletResponse** : cet objet initialise la réponse HTTP qui sera renvoyée au client, et permet de la personnaliser, en initialisant par exemple les en-têtes et le corps (nous verrons comment par la suite)

Servlet

```
package serveletCours;
```

```
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;
```

```
public class ServletFib extends HttpServlet {
    int rangFib;

    public void init(ServletConfig config) throws ServletException{
        super.init(config);
        rangFib=Integer.parseInt(config.getInitParameter("RangFibInitial"));
    }

    public void service(HttpServletRequest req,HttpServletResponse rep) throws
    ServletException,IOException {
        //positionne le type MIME pour la reponse
        rep.setContentType("text/html");
        //recupere une reference vers le flux d'ecriture
        PrintWriter out =rep.getWriter();
        //creation de la page html
        out.println("<HTML>");
        out.println("<head><title>Servlet FIB</title></head>");
        out.println("<body>");
        out.println("<h1>Fib (" +rangFib+") = " + fib(rangFib)+"</h1>");
        out.println("</body></html>");
        //tjr fermer le flux d'ecriture
        out.close();
    }
}
```

Servlet a generalement trois Method:

```
public void init(){}//initialize les param
public void service(){}//traitement
public void destroy(){}//liberation
ressource
```

Les API des servlets

- ☑ Nous allons donc examiner successivement les éléments suivants :
- ☑ L'interface javax.servlet.Servlet qui définit, comme nous l'avons déjà indiqué, le cycle de vie des servlets.
- ☑ L'interface javax.servlet.ServletConfig pour que la servlet puisse accéder à des informations d'initialisation et de contexte d'exécution.
- ☑ La classe javax.servlet.GenericServlet qui implémente l'interface "Servlet" et fournit la structure de base pour la définition de servlets.
- ☑ La classe javax.servlet.http.HttpServlet pour la définition de servlets HTTP
- ☑ L'interface javax.servlet.ServletRequest pour l'envoi de données vers une servlet générique
- ☑ L'interface javax.servlet.http.HttpServletRequest pour l'envoi de données vers une servlet HTTP
- ☑ L'interface javax.servlet.ServletResponse pour la réception de données depuis une servlet générique
- ☑ L'interface javax.servlet.http.HttpServletResponse pour la réception de données depuis une servlet HTTP

```
public static int fib(int n) {
    //traitement suite fibonacci
    //iterative
    int Fn=0;
    int Fn_moins_un=1;
    int Fn_moins_deux=1;
    //0-1-1-2-3
    if(n<2)
        return n;
    for(int i=2;i<n;i++) {
        Fn=Fn_moins_un+
        Fn_moins_deux;
        Fn_moins_deux=Fn_moins_un;
        Fn_moins_un=Fn;
    }
    return Fn;
}
```



Servlet

Les méthodes représentant le cycle de vie d'une

```
public void init(ServletConfig config)
throws ServletException
    • public void service(ServletRequest rq,
        ServletResponse res)
throws ServletException
    • public void destroy()
    • public ServletConfig getServletConfig()
    • public String getServletInfo()
```

L'interface ServletConfig

- Définit les méthodes permettant à une servlet d'accéder aux informations de configuration initiales ainsi qu'à l'environnement d'exécution.
- Deux méthodes d'accès aux paramètres initiaux :
 - getInitParameter (String name)
 - getInitParameterNames
- Une méthode d'accès à l'environnement d'exécution de la servlet
 - getServletContext
- getInitParameter
 - public String getInitParameter(String name)

Retourne une String qui représente la valeur d'un paramètre initial particulier (passer dans le web.xml). Si le paramètre spécifié n'existe pas alors la méthode retourne la valeur null.
- getInitParameterNames
 - public Enumeration getInitParameterNames()

Retourne une énumération qui contient sous forme de String l'ensemble des noms des paramètres initiaux. Les paramètres initiaux sont lus depuis un fichier de propriétés ou le fichier web.xml

```
public void init(ServletConfig config) throws
ServletException {
    // positionne le ServletConfig
    super.init(config);
    // initialise une variable d'état de la servlet
    rangFib =
    Integer.parseInt(config.getInitParameter("Ra
ngFibInitial"));
}
```

```
public void init(ServletConfig config) throws
ServletException {
    // positionne le type MIME pour la réponse
    rep.setContentType("text/html");
    // récupère une référence vers le flux d'écriture
    try (PrintWriter out = rep.getWriter()) {
        // récupère le contenu du formulaire saisi par
        le client
        Enumeration<String> parameters =
        req.getParameterNames();
        while (parameters.hasMoreElements()) {
            String nom = parameters.nextElement();
            String valeur = req.getParameter(nom);
            out.println("Nom du champ : " + nom + " valeur
            :
            " + valeur + "<BR>");
        }
    } catch (IOException e) {
    }
```




Java development kite

JRE:

JVM:

JAR:

variable home:

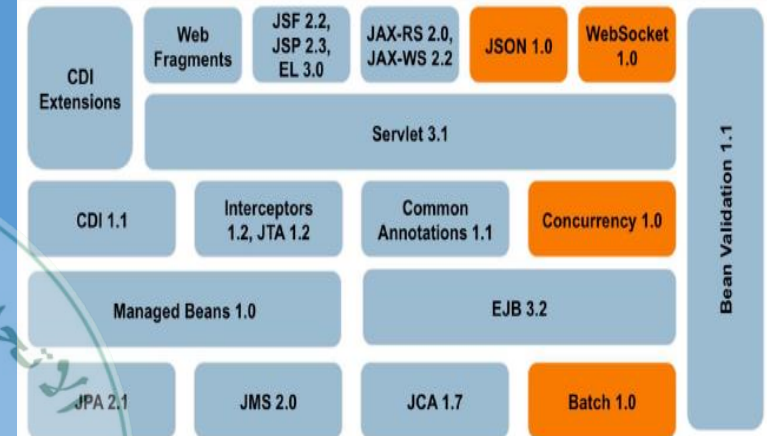
JAVA_HOME

JRE_HOME

Java EE est une extension de la plate-forme standard Java SE, principalement destinée au développement d'applications web.

Un Serveur : C'est un composant que l'on nomme logiquement serveur HTTP. Son travail est simple : il doit écouter tout ce qui arrive sur le port utilisé par le protocole HTTP, le port 80 par exemple, et scruter chaque requête entrante. C'est tout ce qu'il fait, c'est en somme une interface de communication avec le protocole.

Ensemble de composant Java EE



Application Web

- Navigateur Web (Web browser) :
 - Un **programme qui affiche** les fichiers et les **données** disponible sur **Internet**. [The American Heritage Dictionary]
- Serveur Web (Web server) :
 - Un serveur capable de **traiter** des **requête HTTP** envoyées par un client et envoie une réponse **HTTP**.
- Serveur d'application (Application server) :
 - **Sert** du contenu sur le Web mais aussi il supporte différentes fonctionnalités (e.g., Messaging services, Security, etc.)

- Lors de l'utilisation des applications Web, on rencontre, généralement, les étapes suivantes :

Côté Client

Côté Serveur

Côté Client

- Collecte des données utilisateurs (formulaire, cookies, etc.)
- Envoie de la requête au serveur (*requête HTTP*)
- Exécution de la logique métier sur le serveur (*Servlet, PHP, etc.*)
- Accès à une source de données (*Bases de données*)
- Collecte des données à envoyer dans la réponse (*réponse HTTP*)
- Parser et afficher la réponse (*HTML, CSS, etc.*)